09. Pengujian Sistem (*System Testing*)

Mata Kuliah: Pengujian Perangkat Lunak

Program Studi: Teknik Informatika

Fakultas Ilmu Komputer - Universitas Brawijaya

Disusun Oleh: Dr. Achmad Arwan, S.Kom., M.Kom

2026

filkom.ub.ac.id

Capaian Pembelajaran Mata Kuliah

Setelah mengikuti perkuliahan ini, mahasiswa mampu:

- Menjelaskan posisi dan peran System Testing dalam V-Model
- Membedakan konsep **validation** dan **verification**
- Mengidentifikasi artefak yang dibutuhkan dalam pengujian sistem
- Menerapkan teknik black box testing secara tepat
- Menurunkan **use case** → **test scenario** → **test case**
- Merancang pengujian berbasis skenario nyata (real-world)

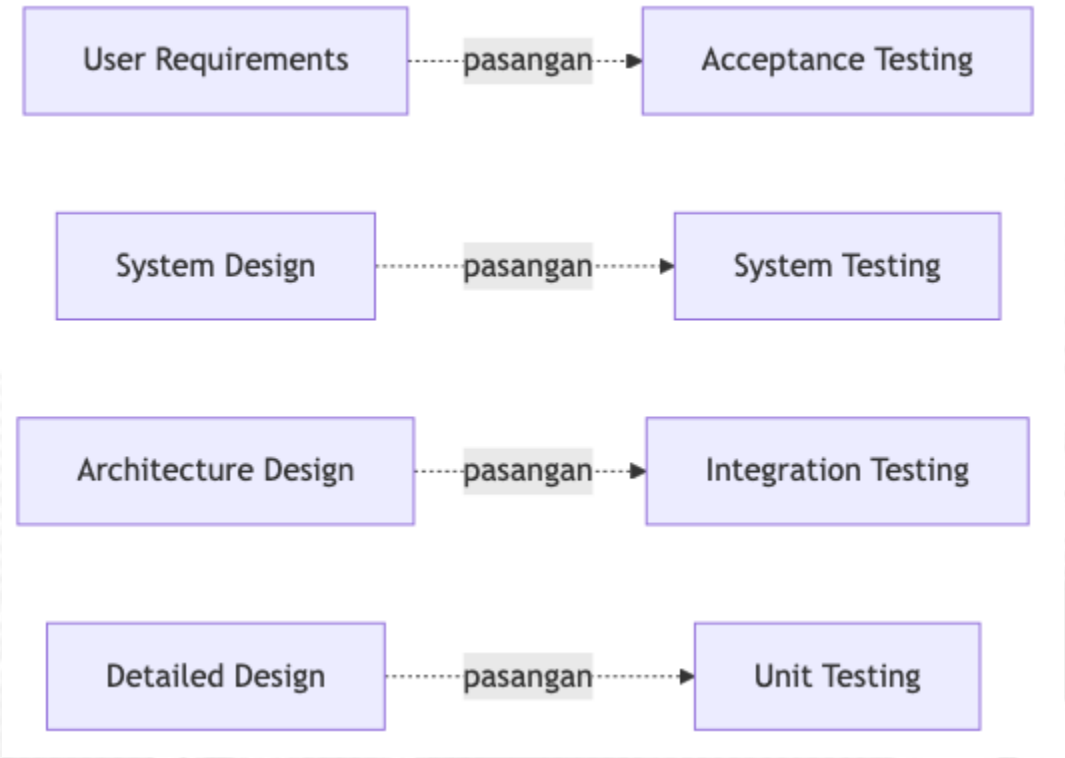
© Fakultas Ilmu Komputer, Universitas Brawijaya

Agenda Perkuliahan

1. Learning Outcomes
2. Posisi *System Testing* di V-Model
3. Definisi & Karakteristik
4. *Validation* vs *Verification*
5. Urgensi & Risiko
6. Contoh Kegagalan Nyata
7. Artefak yang Terlibat
8. *Black Box* & *Spesification-Based Testing*
9. Teknik *Black Box* (5 teknik)
10. *Use Case* → *Test Case*
11. Studi Kasus E-Commerce
12. *Bug* dalam *System Testing*
13. Validasi Sistem (*Deep Insight*)
14. Kesimpulan & Latihan

© Fakultas Ilmu Komputer, Universitas Brawijaya

V-Model: Gambaran Umum



Setiap fase desain di sisi kiri **berpasangan** dengan level pengujian di sisi kanan

Posisi *System Testing* dalam V-Model

- *System Testing* berada di **sisi kanan** V-Model
- Berpasangan dengan ***System Design***
- Input utama: **SRS + System Design Document**
- Dilakukan setelah *Integration Testing* selesai
- Dilakukan sebelum *Acceptance Testing* (UAT)

System Testing memvalidasi bahwa sistem yang telah diintegrasikan sesuai spesifikasi kebutuhan

© Fakultas Ilmu Komputer, Universitas Brawijaya

Traceability dalam V-Model

Fase Kiri (Desain)	Fase Kanan (Pengujian)	Basis Pengujian
User Requirements	Acceptance Testing	User Requirements
System Design	**System Testing**	**SRS**
Architecture Design	Integration Testing	Arsitektur
Detailed Design	Unit Testing	Modul/Fungsi

Traceability: setiap ***requirement*** HARUS dapat ditelusuri ke minimal satu *test case*

© Fakultas Ilmu Komputer, Universitas Brawijaya

Definisi System Testing

> **Formal:** Pengujian pada sistem yang telah terintegrasi secara penuh untuk memverifikasi bahwa sistem memenuhi spesifikasi yang ditetapkan. *(IEEE 829)*

> **Sederhana:** Uji coba sistem dari ujung ke ujung (***end-to-end***) seperti yang akan dilakukan pengguna nyata.

- *Scope*: seluruh sistem, bukan modul individual
- Pelaku: tim QA (independen dari tim developer)
- Basis: SRS / dokumen kebutuhan

© Fakultas Ilmu Komputer, Universitas Brawijaya

Karakteristik System Testing

End-to-End

- Menguji alur lengkap dari input hingga output
- Melibatkan semua komponen berjalan bersama

Black Box Oriented

- Tidak melihat kode internal
- Fokus pada perilaku sistem dari luar

Validation-Oriented

- Menjawab: "*Apakah sistem ini yang diinginkan user?*"
- Berbasis use case & skenario nyata

Requirement-Based

- Setiap test case berasal dari requirement
- RTM memastikan coverage penuh

© Fakultas Ilmu Komputer, Universitas Brawijaya

Validation vs Verification

Aspek	Verification	Validation
Pertanyaan	*"Are we building it right?"*	*"Are we building the right thing?"*
Fokus	Kode, desain, proses	Kebutuhan user, use case
Metode	Review, inspeksi, unit test	System test, UAT
Contoh	Apakah fungsi login sesuai desain?	Apakah login berjalan seperti yang user inginkan?

System Testing adalah pengujian VALIDATION — berbasis kebutuhan pengguna

© Fakultas Ilmu Komputer, Universitas Brawijaya

Urgensi System Testing

Tanpa System Testing:

- Unit test lulus, integrasi OK
- Namun alur bisnis tetap bisa gagal
- Bug tersembunyi di interaksi antar modul
- Requirement tidak tervalidasi
- Sistem "benar teknis" tapi salah perilaku

Dengan System Testing:

- Alur bisnis end-to-end tervalidasi
- Semua requirement diperiksa
- Performa & security diuji
- Risiko kegagalan pasca-rilis diminimalkan
- Kepercayaan stakeholder meningkat

© Fakultas Ilmu Komputer, Universitas Brawijaya

Risiko Jika System Testing Dilewati

- **Kegagalan bisnis:** transaksi tidak konsisten
- **Bug integrasi:** data dari modul A tidak terbaca modul B
- **Security gap:** celah yang tidak terdeteksi unit test
- **Performa buruk:** lambat saat banyak user bersamaan
- **Reputasi rusak:** produk dirilis dengan cacat mayor

85% bug kritis di production sebenarnya dapat dideteksi dengan system testing yang baik

© Fakultas Ilmu Komputer, Universitas Brawijaya

Contoh Kegagalan Nyata

Kasus	Penyebab	Dampak
Therac-25 (1985)	Software tidak diuji secara sistem	6 pasien meninggal akibat overdosis radiasi
Knight Capital (2012)	Deployment bug lolos system test	Rugi $440 juta dalam 45 menit
Healthcare.gov (2013)	System testing tidak memadai	Crash saat launching, diakses jutaan user

Kegagalan sistem skala besar hampir selalu berakar dari system testing yang tidak tuntas

© Fakultas Ilmu Komputer, Universitas Brawijaya

Artefak System Testing: SRS & Test Plan

Software Requirement Specification (SRS)

- Sumber utama semua test
- Berisi functional & non-functional requirement
- Contoh: "Sistem harus memproses checkout dalam < 3 detik"

Test Plan

- Scope: apa yang diuji & tidak diuji
- Strategi: teknik & pendekatan
- Resource: siapa, alat apa
- Jadwal & milestone pengujian
- Kriteria pass/fail

© Fakultas Ilmu Komputer, Universitas Brawijaya

Artefak System Testing: Test Case & Test Data

Test Case

- ID unik (TC-001)
- Deskripsi skenario
- Langkah eksekusi
- Input data
- Expected result
- Actual result & status

Test Data

- Data valid: sesuai format & aturan bisnis
- Data invalid: di luar batas, format salah
- Data boundary: tepat di batas nilai
- Data nyata vs data sintetis

© Fakultas Ilmu Komputer, Universitas Brawijaya

Artefak System Testing: RTM & Test Report

Requirement Traceability Matrix (RTM)

- *Mapping*: *Requirement* ↔ *Test Case*
- Memastikan tidak ada requirement yang terlewat
- Contoh: REQ-05 → TC-012, TC-013, TC-014

Test Report

- Jumlah test case dieksekusi
- Pass / Fail / Blocked
- Defect yang ditemukan
- Coverage requirement
- Rekomendasi go/no-go

RTM adalah bukti bahwa pengujian dilakukan berbasis requirement — bukan ad hoc

TRACEABILITY MATRIX

Requirement Number	Test Case Name
1	. . .
2	
3	. . .
4	
5	. . .
6	. . .
7	. . .
8	. . .

© Fakultas Ilmu Komputer, Universitas Brawijaya

Kenapa System Testing → Black Box?

- System testing menguji **perilaku sistem**, bukan kode
- Penguji tidak perlu (dan tidak harus) tahu implementasi internal
- Fokus: apakah output sesuai dengan requirement?
- Sesuai dengan sudut pandang **pengguna nyata**

Black Box = uji dari luar kotak.
intinya: input → proses (tersembunyi) → **output sesuai ekspektasi**

Termasuk kategori: ***Specification-Based Testing***

© Fakultas Ilmu Komputer, Universitas Brawijaya

Teknik Black Box 1: *Equivalence Partitioning*

- **Membagi input** menjadi kelas-kelas ekuivalen
- Satu representatif dari setiap kelas sudah cukup

Kelas	Contoh (field usia)	Status
Valid	17 – 60 tahun	Valid
Invalid bawah	< 17 tahun	Invalid
Invalid atas	> 60 tahun	Invalid

Efisien: mengurangi jumlah test case tanpa mengurangi coverage

© Fakultas Ilmu Komputer, Universitas Brawijaya

Teknik Black Box 2: Boundary Value Analysis

- Fokus pada **nilai batas** kelas input — di sinilah bug sering muncul
- Uji: nilai minimum, minimum+1, maximum-1, maximum

Batas	Nilai	Yang Diuji
Min	17	Tepat batas bawah
Min+1	18	Satu di atas batas bawah
Max-1	59	Satu di bawah batas atas
Max	60	Tepat batas atas

*BVA sering **dikombinasikan** dengan Equivalence Partitioning*

© Fakultas Ilmu Komputer, Universitas Brawijaya

Teknik Black Box 3: Decision Table Testing

- Digunakan saat ada **kombinasi kondisi** yang menghasilkan aksi berbeda

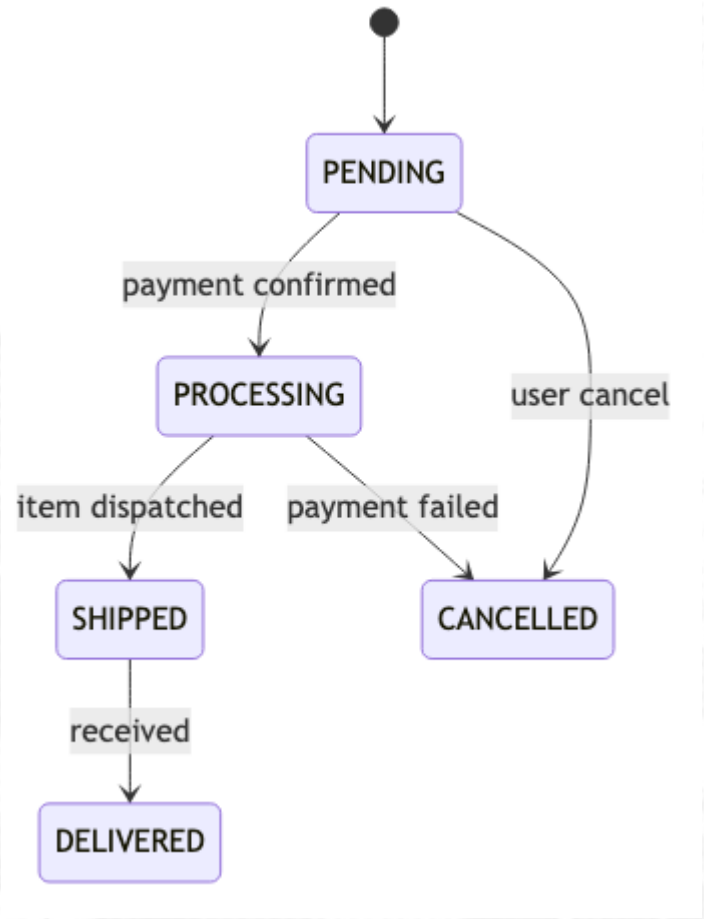
Kondisi / Aksi	Kasus 1	Kasus 2	Kasus 3	Kasus 4
Login valid?	Y	Y	N	N
Email verified?	Y	N	Y	N
Aksi: Boleh masuk	✅	❌	❌	❌
Aksi: Kirim verifikasi	❌	✅	❌	❌

Berguna untuk menghindari missed combination

© Fakultas Ilmu Komputer, Universitas Brawijaya

Teknik Black Box 4: State Transition Testing

- Menguji **perubahan state** sistem berdasarkan event



Penting untuk sistem order, booking, workflow approval

Teknik Black Box 5: Use Case Testing

- Menurunkan test case langsung dari **use case scenario**
- Menutup gap antara requirement dan pengujian
- Menjamin validasi dari perspektif **pengguna nyata**

Langkah:

1. Ambil use case dari SRS
2. Identifikasi main flow & alternative flow
3. Turunkan setiap flow menjadi test scenario
4. Dari scenario → buat test case detail

> *Use Case Testing* adalah **teknik paling strategis** di system testing

© Fakultas Ilmu Komputer, Universitas Brawijaya

Ringkasan Teknik Black Box

Teknik	Kapan Digunakan	Kekuatan
Equivalence Partitioning	Input berupa rentang nilai	Efisien, mengurangi jumlah TC
Boundary Value Analysis	Ada nilai batas input	Deteksi off-by-one error
Decision Table	Banyak kondisi & kombinasi	Tidak ada kombinasi terlewat
State Transition	Sistem punya state & transisi	Uji alur state lengkap
Use Case Testing	Semua skenario user	Validasi requirement nyata

© Fakultas Ilmu Komputer, Universitas Brawijaya

Validasi Berbasis Use Case: Konsep

System testing = validasi requirement → semua test harus berakar dari **use case**

Jika test case **tidak dapat ditelusuri** ke requirement/use case, maka test tersebut **tidak bernilai** dalam system testing

Langkah: *Use Case → Test Case*

1. **Ambil use case** dari SRS atau dokumen analisis
2. **Identifikasi actor** — siapa yang melakukan aksi
3. **Petakan main flow** — alur normal (happy path)
4. **Petakan alternative flow** — kondisi abnormal, error, batas
5. **Turunkan jadi test scenario** — kalimat skenario uji
6. **Turunkan jadi test case** — langkah, input, expected result
7. **Petakan ke RTM** — requirement → test case

© Fakultas Ilmu Komputer, Universitas Brawijaya

Studi Kasus: E-Commerce Checkout System

Sistem: Platform belanja online

Use Case: Checkout & Pembayaran

Actor: Pelanggan (Customer)

Precondition:

- Customer sudah login
- Minimal 1 item di keranjang
- Stok produk tersedia

Postcondition:

- Order berhasil dibuat dengan status PENDING
- Stok produk berkurang
- Notifikasi dikirim ke customer & merchant

© Fakultas Ilmu Komputer, Universitas Brawijaya

Main Flow: Checkout (8 Langkah)

1. Customer membuka halaman keranjang belanja
2. Customer memilih item dan mengklik **"Checkout"**
3. Sistem menampilkan form pengiriman & ringkasan order
4. Customer mengisi alamat pengiriman dan memilih kurir
5. Customer memilih metode pembayaran (transfer/e-wallet/kartu)
6. Customer mengklik **"Bayar Sekarang"**
7. Sistem memproses pembayaran via payment gateway
8. Sistem membuat order, mengurangi stok, mengirim notifikasi → tampilkan halaman sukses

Alternative Flow 1 & 2

AF-1: Stok Habis

- Di langkah 3, sistem cek stok
- Jika stok = 0 saat checkout → tampilkan pesan "Stok tidak tersedia"
- Customer diarahkan kembali ke keranjang
- Item otomatis ditandai "Habis"

AF-2: Pembayaran Gagal

- Di langkah 7, payment gateway mengembalikan kode gagal
- Sistem menampilkan pesan "Pembayaran gagal"
- Order tidak dibuat, stok tidak berkurang
- Customer ditawarkan opsi coba lagi atau ganti metode

© Fakultas Ilmu Komputer, Universitas Brawijaya

Alternative Flow 3 & 4

AF-3: Input Alamat Tidak Valid

- Di langkah 4, customer tidak mengisi alamat lengkap
- Sistem menampilkan validasi per field yang kosong
- Proses tidak dapat dilanjutkan sampai form valid
- Customer harus mengisi ulang data yang kurang

AF-4: Timeout API Payment Gateway

- Di langkah 7, payment gateway tidak merespons dalam 30 detik
- Sistem menampilkan pesan "Koneksi bermasalah, silakan coba lagi"
- Status order = UNKNOWN → sistem trigger proses reconciliation
- Customer mendapat email konfirmasi status dalam 1x24 jam

© Fakultas Ilmu Komputer, Universitas Brawijaya

Test Scenario: Checkout System

No	Skenario	Jenis	Teknik
TS-01	Checkout sukses — semua data valid	Positive	Use Case
TS-02	Checkout gagal — stok habis	Negative	State Transition
TS-03	Checkout gagal — pembayaran ditolak	Negative	Use Case + Decision Table
TS-04	Form alamat tidak diisi	Boundary/Error	Equivalence Partitioning
TS-05	Timeout API lebih dari 30 detik	Error Scenario	Boundary Value

© Fakultas Ilmu Komputer, Universitas Brawijaya

Test Case 1 & 2

Field	TC-001	TC-002
Skenario	Checkout sukses	Checkout — stok habis
Input	Item valid, data lengkap, saldo cukup	Item stok = 0
Langkah	Login → cart → checkout → bayar	Login → cart → checkout
Expected	Order dibuat, status PENDING, notif terkirim	Pesan "Stok tidak tersedia", redirect ke cart
Teknik	Use Case Testing	State Transition
Status	**PASS**	**PASS**

Test Case 3 & 4

Field	TC-003	TC-004
Skenario	Pembayaran ditolak gateway	Form alamat kosong
Input	Kartu kredit expired	Field alamat dibiarkan kosong
Langkah	Login → cart → checkout → bayar	Login → cart → checkout → klik lanjut
Expected	Pesan "Pembayaran gagal", order tidak dibuat	Validasi form aktif, field wajib ditandai merah
Teknik	Decision Table	Equivalence Partitioning
Status	**PASS**	**PASS**

Test Case 5

Field	Detail
ID	TC-005
Skenario	Timeout API payment gateway
Input	Simulasi response gateway > 30 detik
Langkah	Login → cart → checkout → bayar → tunggu timeout
Expected	Pesan "Koneksi bermasalah", status order = UNKNOWN, email dikirim
Teknik	Boundary Value Analysis (batas waktu 30 detik)
Status	**PASS**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Mapping Test Case → Teknik Black Box

Test Case	Teknik Utama	Reasoning
TC-001	Use Case Testing	Main flow checkout persis dari use case
TC-002	State Transition	State order berubah: CART → BLOCKED (stok 0)
TC-003	Decision Table	Kombinasi: kartu valid/invalid × saldo cukup/tidak
TC-004	Equivalence Partitioning	Field kosong = kelas invalid
TC-005	Boundary Value Analysis	Tepat di batas waktu 30 detik

© Fakultas Ilmu Komputer, Universitas Brawijaya

Reasoning Pemilihan Teknik

- **Use Case Testing** → alur bisnis utama, multi-langkah, berbasis requirement
- **State Transition** → sistem order memiliki state (PENDING, PROCESSING, SHIPPED, CANCELLED)
- **Decision Table** → pembayaran punya banyak kombinasi kondisi (metode × status × saldo)
- **Equivalence Partitioning** → validasi form punya kelas: kosong, format salah, valid
- **Boundary Value** → timeout punya nilai batas: < 30 detik = lanjut, ≥ 30 detik = error

Tips: Pilih teknik berdasarkan karakteristik input & perilaku sistem — bukan kebiasaan

© Fakultas Ilmu Komputer, Universitas Brawijaya

Bug 1: *Race Condition*

Skenario:

- User A & User B checkout item yang sama (stok = 1) secara bersamaan

Yang terjadi tanpa system testing:

- Kedua user mendapat konfirmasi sukses
- Stok menjadi -1 → data inkonsisten

Cara deteksi di system testing:

- Test scenario: dua session checkout bersamaan
- Verifikasi: sistem harus handle concurrent request dengan locking/queue

> *Race condition* jarang muncul di unit test — hanya terdeteksi di system testing dengan concurrent scenario

© Fakultas Ilmu Komputer, Universitas Brawijaya

Bug 2 & 3

Payment Inconsistency

- Payment gateway konfirmasi sukses
- Tapi order di sistem = FAILED
- Akibat: customer sudah bayar tapi order gagal
- Deteksi: test scenario timeout + cek status sinkronisasi antar sistem

Data Mismatch

- Harga di cart berbeda dengan harga di konfirmasi order
- Disebabkan: cache harga tidak diupdate saat flash sale
- Akibat: customer bayar harga lama, merchant merugi
- Deteksi: test dengan perubahan harga saat proses checkout aktif

© Fakultas Ilmu Komputer, Universitas Brawijaya

Validasi Sistem: Apa yang Divalidasi?

System testing memvalidasi lebih dari sekadar fungsi:

Dimensi Validasi	Contoh
Validasi Bisnis	Alur order sesuai proses bisnis nyata
Validasi Data	Data tersimpan benar, konsisten antar modul
Validasi Flow	Urutan langkah sesuai use case
Validasi State	Transisi status order (PENDING → SHIPPED) benar
Validasi Performa	Checkout selesai < 3 detik di load normal
Validasi Security	Session tidak bocor antar user

Validasi Bisnis & Data (Deep Dive)

Validasi Bisnis

- Apakah aturan bisnis ditegakkan sistem?
- Contoh: diskon hanya berlaku 1x per user
- Contoh: minimum order terpenuhi sebelum checkout
- Contoh: stok tidak bisa negatif

Validasi Data

- Data tersimpan lengkap di database
- Relasi antar tabel konsisten
- Data yang dikirim = data yang disimpan
- Tidak ada data hilang di tengah transaksi (atomicity)

© Fakultas Ilmu Komputer, Universitas Brawijaya

Validasi Flow & State (Deep Dive)

Validasi Flow

- Langkah tidak bisa dilewati sembarangan
- Contoh: tidak bisa ke halaman pembayaran tanpa isi alamat
- Contoh: tidak bisa checkout keranjang kosong
- Contoh: redirect benar setelah setiap aksi

Validasi State

- State tidak bisa lompat sembarangan
- Order DELIVERED tidak bisa kembali ke PENDING
- Order CANCELLED tidak bisa diproses ulang
- Setiap transisi state punya trigger yang benar

© Fakultas Ilmu Komputer, Universitas Brawijaya

Common Mistake ⚠️

Dalam desain test case:

- Test case tidak berasal dari requirement
- Hanya menguji “*happy path*”
- Test case duplikat — tidak ada nilai tambah
- Lupa alternative flow & error scenario

Dalam pemilihan teknik:

- Semua test case pakai Use Case Testing saja
- Tidak tahu kapan pakai BVA vs EP
- Tidak mengisi RTM (claim "sudah menguji semua")
- Expected result tidak spesifik: "tidak error"

"***Sudah coba, tidak error***" bukan test case — expected result harus terukur dan spesifik

© Fakultas Ilmu Komputer, Universitas Brawijaya

Kesimpulan

- System Testing = **level pengujian tertinggi sebelum UAT**
- Berpasangan dengan **System Design** dalam V-Model
- Merupakan aktivitas **VALIDATION** — berbasis kebutuhan user
- Teknik utama: **black box** (EP, BVA, Decision Table, State Transition, Use Case)
- Kunci sukses: **use case** → **test scenario** → **test case** → **RTM**

> Sistem bisa benar secara teknis, tetapi tetap gagal jika perilakunya tidak sesuai kebutuhan user

© Fakultas Ilmu Komputer, Universitas Brawijaya

Latihan Mandiri: System Testing Design

Sistem: Aplikasi Pemesanan Makanan Online

Instruksi:

1. Buat **minimal 2 use case** (contoh: login, pesan makanan)
2. Setiap use case: main flow (min. 5 langkah) + min. 2 alternative flow
3. Turunkan ke **minimal 3 test case per use case** (positive, negative, boundary)
4. Petakan setiap test case ke **teknik black box** yang tepat beserta reasoning
5. Buat **RTM sederhana:** mapping requirement → test case

© Fakultas Ilmu Komputer, Universitas Brawijaya

LAtihan: Format Output

Bagian	Isi Wajib
Use Case	Nama, actor, precondition, main flow, alternative flow
Test Scenario	Daftar skenario per use case
Test Case	ID, skenario, input, langkah, expected result, teknik
Mapping Teknik	Teknik yang digunakan + alasan pemilihan
RTM	Requirement ID → Test Case ID

© Fakultas Ilmu Komputer, Universitas Brawijaya

Referensi

- Sommerville, I. (2016). *Software Engineering*, 10th Edition. Pearson
- Pressman, R.S. (2014). *Software Engineering: A Practitioner's Approach*, 8th Ed
- ISTQB Foundation Level Syllabus v4.0: https://www.istqb.org
- IEEE 829 — Standard for Software Test Documentation
- Myers, G.J. (2011). *The Art of Software Testing*, 3rd Edition
- Guru99 Software Testing: https://www.guru99.com/software-testing.html

© Fakultas Ilmu Komputer, Universitas Brawijaya